2. Apache Spark and RDDs

2.1 Introduction

Philosophy

`Spark` computing framework deals with many complex issues: fault tolerance, slow machines, big datasets, etc.

"Here's an operation, run it on all the data."

- I do not care where it runs
- Feel free to run it twice on different nodes

Jobs are divided in subtasks, that are executed by the workers

- How do we deal with **failure**? Launch **another task**!
- How do we deal with **stragglers**? Launch **another task**!
  (and kill original task)

API

An API allows a user to interact with the software

`Spark` is implemented in **Scala**, runs on the **JVM** (Java Virtual Machine)

Multiple Application Programming Interfaces (APIs):

- `Scala` (JVM)
- `Java` (JVM)
- `Python`
- `R`

This course uses the Python API. Easier to learn than `Scala` and `Java`

- About the `R` API: **still young** and outlying because of `R` syntax.

Architecture

When you interact with `Spark` through its API, you **send instructions to the Driver**

- The **Driver** is the **central coordinator**
- It **communicates with distributed workers** called **executors**
- Creates a **logical directed acyclic graph** (DAG) of operations
- **Merges operations** that can be merged
- **Splits** the operations in **tasks** (smallest unit of work in Spark)
- **Schedules** the tasks and **send them to the executors**
- **Tracks** data and tasks

Example

- Example of DAG: `map(f) - map(g) - filter(h) - reduce(l)`
- `map(f o g)`

SparkContext object

You interact with the **driver** through the `SparkContext` object.

- **In the `Spark interactive shell`**
  `SparkContext` is automatically created in the and named `sc`

- **In a `jupyter notebook`**
  Create a `SparkContext` object using:

```
>>> from pyspark import SparkConf, SparkContext

>>> conf = SparkConf().setAppName(appName).setMaster(master)
>>> sc = SparkContext(conf=conf)
```

RDDs and running model

Spark programs are written in terms of operations on **RDDs**

- RDD = **Resilient Distributed Dataset**
- An **immutable distributed collection** of objects spread across the cluster disks or memory
- RDDs can contain any type of Python, Java, or Scala objects, including user-defined classes
- Parallel **transformations** and **actions** can be applied to RDDs
- RDDs are automatically rebuilt on machine failure

Creating a RDD

From an iterable object `iterator` (e.g. a `Python list`, etc.):

```
lines = sc.parallelize(iterator)
```

From a text file:

```
lines = sc.textFile("/path/to/file.txt")
```

where `lines` is the resulting RDD, and `sc` the spark context

Remarks

- `parallelize` not really used in practice
- In real life: **load data from external storage**
- External storage is often **HDFS** (Hadoop Distributed File System)
- Can read most formats (`json`, `csv`, `xml`, `parquet`, `orc`, etc.)

Operations on RDD

Two families of operations can be performed on RDDs

- **Transformations**
  Operations on RDDs which return a new RDD
  Lazy evaluation

- **Actions**
  Operations on RDDs that return some other data type
  Triggers computations

What is lazy evaluation ? When a transformation is called on an RDD:

- The operation is **not immediately performed**
- Spark internally **records that this operation has been requested**
- Computations are triggered only **if an action requires the result of this transformation** at some point

2.2 Transformations

Transformations

The most important transformation is `map`

transformation	description
`map(f)`	apply a function `f` to each element of the RDD

Here is an example:

```
>>> rdd = sc.parallelize([2, 3, 4])
>>> rdd.map(lambda x: list(range(1, x))).collect()
[[1], [1, 2], [1, 2, 3]]
```

- We need to call `collect` (an **action**) otherwise **nothing happens**
- Once again, `map` is lazy-evaluated
- In `Python`, **three options for passing functions** into Spark
- For short functions: `lambda` expressions (anonymous functions)
- Otherwise: top-level functions or **locally defined functions** with `def`

Transformations

About passing functions to `map`:

- Involves serialization with `pickle`
- `Spark` sends the **entire pickled function** to worker nodes

Warning. If the function is an **object method**:

- The **whole object is pickled** since the method contains references to the object (`self`) and references to attributes of the object
- The whole object can be **large**
- The whole object **may not be serializable with `pickle`**

`[Let's go to notebook05_sparkrdd.ipynb]`

Transformations

Then we have `flatMap`

transformation	description
`flatMap(f)`	apply `f` to each element of the RDD, then flattens the results

Example

```
>>> rdd = sc.parallelize([2, 3, 4, 5])
>>> rdd.flatMap(lambda x: range(1, x)).collect()
[1, 1, 2, 1, 2, 3, 1, 2, 3, 4]
```

Transformations

`filter` allows to filter an RDD

transformation	description
`filter(f)`	Return an RDD consisting of only elements that pass the condition `f` passed to `filter()`

Example

```
>>> rdd = sc.parallelize(range(10))
>>> rdd.filter(lambda x: x % 2 == 0).collect()
[0, 2, 4, 6, 8]
```

Transformations

About `distinct` and `sample`

transformation	description
`distinct()`	Removes duplicates
`sample(withReplacement, fraction, [seed])`	Sample an RDD, with or without replacement

Example

```
>>> rdd = sc.parallelize([1, 1, 4, 2, 1, 3, 3])
>>> rdd.distinct().collect()
[1, 2, 3, 4]
```

Transformations

We have also "pseudo set" operations

transformation	description
`union(otherRdd)`	Returns union with `otherRdd`
`instersection(otherRdd)`	Returns intersection with `otherRdd`
`subtract(otherRdd)`	Return each value in `self` that is not contained in `otherRdd`.

- If if there are duplicates in the input RDD, the result of `union()` **will contain duplicates** (fixed with `distinct()`)
- `intersection()` removes all duplicates (including duplicates from a single RDD)
- Performance of `intersection()` is much worse than `union()` since it requires a **shuffle** to identify common elements
- `subtract` also requires a shuffle

Transformations

We have also "pseudo set" operations

transformation	description
`union(otherRdd)`	Returns union with `otherRdd`
`instersection(otherRdd)`	Returns intersection with `otherRdd`
`subtract(otherRdd)`	Return each value in `self` that is not contained in `otherRdd`.

Example with `union` and `distinct`

```
>>> rdd1 = sc.parallelize(range(5))
>>> rdd2 = sc.parallelize(range(3, 9))
>>> rdd3 = rdd1.union(rdd2)
>>> rdd3.collect()
[0, 1, 2, 3, 4, 3, 4, 5, 6, 7, 8]
```

```
>>> rdd3.distinct().collect()
[0, 1, 2, 3, 4, 5, 6, 7, 8]
```

About shuffles

- Certain operations trigger a **shuffle**
- It is Spark's mechanism for **re-distributing data** so that it's grouped differently across partitions
- It involves **copying data across executors and machines**, making the shuffle a complex and costly operation
- We will discuss shuffles in detail later in the course

Performance Impact

- A shuffle involves disk I/O, data serialization and network I/O.
- To organize data for the shuffle, Spark generates sets of **tasks**
  - **map tasks** to organize the data
  - and a set of **reduce tasks** to aggregate it. This nomenclature comes from MapReduce and does not directly relate to Spark's map and reduce operations.

Transformations

Another "pseudo set" operation

transformation	description
`cartesian(otherRdd)`	Return the Cartesian product of this RDD and another one

Example

```
>>> rdd1 = sc.parallelize([1, 2])
>>> rdd2 = sc.parallelize(["a", "b"])
>>> rdd1.cartesian(rdd2).collect()
[(1, 'a'), (1, 'b'), (2, 'a'), (2, 'b')]
```

- `cartesian()` is **very expensive** for large RDDs

`[Let's go to notebook05_sparkrdd.ipynb]`

2.3 Actions

Actions

`collect` brings the `RDD` back to the driver

transformation	description
`collect()`	Return all elements from the RDD

Example

```
>>> rdd = sc.parallelize([1, 2, 3, 3])
>>> rdd.collect()
[1, 2, 3, 3]
```

Remarks

- Be sure that the **retrieved data fits in the driver memory** !
- Useful when developping and working on small data for testing
- We'll use it a lot here, but **we don't use it in real-world problems**

Actions

It's important to count !

transformation	description
`count()`	Return the number of elements in the RDD
`countByValue()`	Return the count of each unique value in the RDD as a dictionary of `{value: count}` pairs.

Example

```
>>> rdd = sc.parallelize([1, 3, 1, 2, 2, 2])
>>> rdd.count()
6
```

```
>>> rdd.countByValue()
defaultdict(int, {1: 2, 3: 1, 2: 3})
```

Actions

How to get some values in an RDD ?

action	description
`take(n)`	Return `n` elements from the RDD (deterministic)
`top(n)`	Return first `n` elements from the RDD (decending order)
`takeOrdered(num, key=None)`	Get the N elements from a RDD ordered in ascending order or as specified by the optional key function.

Remarks

- `take(n)` returns n elements from the RDD and attempts to **minimize the number of partitions it accesses**
- So, it may represent a **biased** collection
- `collect` and `take` may not return the elements in the order you might expect

Actions

How to get some values in an RDD ?

action	description
`take(n)`	Return `n` elements from the RDD (deterministic)
`top(n)`	Return first `n` elements from the RDD (decending order)
`takeOrdered(num, key=None)`	Get the N elements from a RDD ordered in ascending order or as specified by the optional key function.

Example

```
>>> rdd = sc.parallelize([(3, 'a'), (1, 'b'), (2, 'd')])
>>> rdd.takeOrdered(2)
[(1, 'b'), (2, 'd')]
```

```
>>> rdd.takeOrdered(2, key=lambda x: x[1])
[(3, 'a'), (1, 'b')]
```

Actions

The `reduce` action

action	description
`reduce(f)`	Reduces the elements of this RDD using the specified commutative and associative binary operator `f`.
`fold(zeroValue, op)`	Same as reduce() but with the provided zero value.

- `op(x, y)` is allowed to modify x and return it as its result value to avoid object allocation; however, it should not modify y.
- `reduce` applies some operation to pairs of elements until there is just one left. Throws an exception for empty collections.
- `fold` has initial zero-value: defined for empty collections.

Actions

The `reduce` action

action	description
`reduce(f)`	Reduces the elements of this RDD using the specified commutative and associative binary operator `f`.
`fold(zeroValue, op)`	Same as reduce() but with the provided zero value.

Example

```
>>> rdd = sc.parallelize([1, 2, 3])
>>> rdd.reduce(lambda a, b: a + b)
6
```

```
>>> rdd.fold(0, lambda a, b: a + b)
6
```

Actions

The `reduce` action

action	description
`reduce(f)`	Reduces the elements of this RDD using the specified commutative and associative binary operator `f`.
`fold(zeroValue, op)`	Same as reduce() but with the provided zero value.

Warning with `fold`. Solutions can depend on the number of partitions

```
>>> rdd = sc.parallelize([1, 2, 4], 2) # RDD with 2 partitions
>>> rdd.fold(2.5, lambda a, b: a + b)
14.5
```

- RDD has 2 partition: say [1, 2] and [4]
- Sum in the partitions: 2.5 + (1 + 2) = 5.5 and 2.5 + (4) = 6.5
- Sum over partitions: 2.5 + (5.5 + 6.5) = 14.5

Actions

The `reduce` action

action	description
`reduce(f)`	Reduces the elements of this RDD using the specified commutative and associative binary operator `f`.
`fold(zeroValue, op)`	Same as reduce() but with the provided zero value.

Warning with `fold`. Solutions can depend on the number of partitions

```
>>> rdd = sc.parallelize([1, 2, 3], 5) # RDD with 5 partitions
>>> rdd.fold(2, lambda a, b: a + b)
???
```

[Let's go to notebook05_sparkrdd.ipynb]

Actions

The `reduce` action

action	description
`reduce(f)`	Reduces the elements of this RDD using the specified commutative and associative binary operator `f`.
`fold(zeroValue, op)`	Same as reduce() but with the provided zero value.

Warning with `fold`. Solutions can depend on the number of partitions

```
>>> rdd = sc.parallelize([1, 2, 3], 5) # RDD with 5 partitions
>>> rdd.fold(2, lambda a, b: a + b)
18
```

- Yes, even if there is less partitions than elements !
- 18 = 2 * 5 + (1+2+3) + 2

Actions

The `aggregate` action

action	description
`aggregate(zero, seqOp, combOp)`	Similar to reduce() but used to return a different type.

Aggregates the elements of each partition, and then the results for all the partitions, given aggregation functions and zero value.

- `seqOp(acc, val)`: function to combine the elements of a partition from the RDD (`val`) with an accumulator (`acc`). It can return a different result type than the type of this `RDD`
- `combOp`: function that merges the accumulators of two partitions
- Once again, in both functions, the first argument can be modified while the second cannot

Actions

The `aggregate` action

action	description
`aggregate(zero, seqOp, combOp)`	Similar to reduce() but used to return a different type.

Example

```
>>> seqOp = lambda x, y: (x[0] + y, x[1] + 1)
>>> combOp = lambda x, y: (x[0] + y[0], x[1] + y[1])
>>> sc.parallelize([1, 2, 3, 4]).aggregate((0, 0), seqOp, combOp)
(10, 4)
```

```
>>> sc.parallelize([]).aggregate((0, 0), seqOp, combOp)
(0, 0)
```

[Let's go to notebook05_sparkrdd.ipynb]

Actions

The `foreach` action

action	description
`foreach(f)`	Apply a function `f` to each element of a RDD

- Performs an action on all of the elements in the RDD without returning any result to the driver.
- Example : insert records into a database with `f`

The `foreach()` action lets us perform computations on each element in the RDD without bringing it back locally

2.4 Persistence

Lazy evaluation and persistence

- Spark RDDs are **lazily evaluated**
- Each time an action is called on a RDD, this RDD and all its dependencies are **recomputed**
- If you plan to reuse a RDD multiple times, you should use **persistence**

Remarks

- Lazy evaluation helps `spark` to **reduce the number of passes** over the data it has to make by grouping operations together
- No substantial benefit to writing a single complex map instead of chaining together many simple operations
- Users are free to organize their program into **smaller**, more **manageable operations**

Persistence

How to use persistence ?

method	description
`cache()`	Persist the RDD in memory
`persist(storageLevel)`	Persist the RDD according to `storageLevel`

- These methods must be called **before** the action, and do not trigger the computation

Usage of `storageLevel`

```
pyspark.StorageLevel(
  useDisk, useMemory, useOffHeap, deserialized, replication=1
)
```

Persistence

Options for persistence

argument	description
`useDisk`	Allow caching to use disk if `True`
`useMemory`	Allow caching to use memory if `True`
`useOffHeap`	Store data outside of JVM heap if `True`. Useful if using some in-memory storage system (such a `Tachyon`)
`deserialized`	Cache data without serialization if `True`
`replication`	Number of replications of the cached data

Persistence

Options for persistence

argument	description
`useDisk`	Allow caching to use disk if `True`
`useMemory`	Allow caching to use memory if `True`
`useOffHeap`	Store data outside of JVM heap if `True`. Useful if using some in-memory storage system (such a `Tachyon`)
`deserialized`	Cache data without serialization if `True`
`replication`	Number of replications of the cached data

`replication`

- If you cache data that is quite slow to be recomputed, you can use replications. If a machine fails, data will not have to be recomputed.

Persistence

Options for persistence

argument	description
`useDisk`	Allow caching to use disk if `True`
`useMemory`	Allow caching to use memory if `True`
`useOffHeap`	Store data outside of JVM heap if `True`. Useful if using some in-memory storage system (such a `Tachyon`)
`deserialized`	Cache data without serialization if `True`
`replication`	Number of replications of the cached data

`deserialized`

- Serialization is conversion of the data to a binary format
- To the best of our knowledge, `PySpark` only support serialized caching (using `pickle`)

Persistence

Options for persistence

argument	description
`useDisk`	Allow caching to use disk if `True`
`useMemory`	Allow caching to use memory if `True`
`useOffHeap`	Store data outside of JVM heap if `True`. Useful if using some in-memory storage system (such a `Tachyon`)
`deserialized`	Cache data without serialization if `True`
`replication`	Number of replications of the cached data

`useOffHeap`

- Data cached in the JVM heap by default
- Very interesting alternative in-memory solutions such as `tachyon`
- Don't forget that `spark` is `scala` running on the JVM

Back to options for persistence

```
StorageLevel(useDisk, useMemory, useOffHeap, deserialized, replication)
```

You can use these constants:

```
DISK_ONLY = StorageLevel(True, False, False, False, 1)
DISK_ONLY_2 = StorageLevel(True, False, False, False, 2)
MEMORY_AND_DISK = StorageLevel(True, True, False, True, 1)
MEMORY_AND_DISK_2 = StorageLevel(True, True, False, True, 2)
MEMORY_AND_DISK_SER = StorageLevel(True, True, False, False, 1)
MEMORY_AND_DISK_SER_2 = StorageLevel(True, True, False, False, 2)
MEMORY_ONLY = StorageLevel(False, True, False, True, 1)
MEMORY_ONLY_2 = StorageLevel(False, True, False, True, 2)
MEMORY_ONLY_SER = StorageLevel(False, True, False, False, 1)
MEMORY_ONLY_SER_2 = StorageLevel(False, True, False, False, 2)
OFF_HEAP = StorageLevel(False, False, True, False, 1)
```

and simply call for instance

```
rdd.persist(MEMORY_AND_DISK)
```

Persistence

What if you attempt to **cache too much data to fit in memory ?**

Spark will automatically evict old partitions using a Least Recently Used (LRU) cache policy:

- For the **memory-only** storage levels, it will recompute these partitions the next time they are accessed
- For the **memory-and-disk** ones, it will write them out to disk

Use `unpersist()` to RDDs to **manually remove them** from the cache

Reminder: about passing functions (again)

Warning

- When passing functions, you can **inadvertently serialize the object containing the function**.

If you pass a function that:

- is the member of an object
- contains references to fields in an object

then `Spark` sends the **entire object to worker nodes**, which can be **much larger** than the bit of information you need

- This can cause your **program to fail**, if your class contains objects that **Python can't pickle**

About passing functions

Passing a function with field references (**don't do this !**)

```
class SearchFunctions(object):

  def __init__(self, query):
      self.query = query

  def isMatch(self, s):
      return self.query in s

  def getMatchesFunctionReference(self, rdd):
      # Problem: references all of "self" in "self.isMatch"
      return rdd.filter(self.isMatch)

  def getMatchesMemberReference(self, rdd):
      # Problem: references all of "self" in "self.query"
      return rdd.filter(lambda x: self.query in x)
```

Instead, **just extract the fields you need** from your object into a local variable and pass that in

About passing functions

Python function passing without field references

```
class WordFunctions(object):
  ...

def getMatchesNoReference(self, rdd):
  # Safe: extract only the field we need into a local variable
  query = self.query
  return rdd.filter(lambda x: query in x)
```

Much better to do this instead

2.5. Pair RDD: key-value pairs

Pair RDD: key-value pairs

It's roughly an RDD where each element is a tuple with two elements: a key and a value

- For numerous tasks, such as aggregations tasks, storing information as `(key, value)` pairs into RDD is very convenient
- Such RDDs are called `PairRDD`
- Pair RDDs expose **new operations** such as **grouping together** data with the same key, and **grouping together two different RDDs**

Creating a pair RDD

Calling `map` with a function returning a `tuple` with two elements

```
>>> rdd = sc.parallelize([[1, "a", 7], [2, "b", 13], [2, "c", 17]])
>>> rdd = rdd.map(lambda x: (x[0], x[1:]))
>>> rdd.collect()
[(1, ['a', 7]), (2, ['b', 13]), (2, ['c', 17])]
```

Warning

An element mut be tuples with two elements (the key and the value)

```
>>> rdd = sc.parallelize([[1, "a", 7], [2, "b", 13], [2, "c", 17]])
>>> rdd.keys().collect()
[1, 2, 2]
>>> rdd.values().collect()
['a', 'b', 'c']
```

For things to work as expected you **must** do

```
>>> rdd = sc.parallelize([[1, "a", 7], [2, "b", 13], [2, "c", 17]])\
      .map(lambda x: (x[0], x[1:]))
>>> rdd.keys().collect()
[1, 2, 2]
>>> rdd.values().collect()
[['a', 7], ['b', 13], ['c', 17]]
```

Transformations for a single `PairRDD`

transformation	description
`keys()`	Return an RDD containing the keys.
`values()`	Return an RDD containing the values.
`sortByKey()`	Return an RDD sorted by the key.
`mapValues(f)`	Apply a function `f` to each value of a pair RDD without changing the key.

Example with `mapValues`

```
>>> rdd = sc.parallelize([("a", "x y z"), ("b", "p r")])
>>> rdd.mapValues(lambda v: v.split(' ')).collect()
[('a', ['x', 'y', 'z']), ('b', ['p', 'r'])]
```

Transformations for a single `PairRDD`

transformation	description
`flatMapValues(f)`	Pass each value in the key-value pair RDD through a flatMap function `f` without changing the keys.

Example with `flatMapValues`

```
>>> texts = sc.parallelize([("a", "x y z"), ("b", "p r")])
>>> tokenize = lambda x: x.split(" ")
>>> texts.flatMapValues(tokenize).collect()
[('a', 'x'), ('a', 'y'), ('a', 'z'), ('b', 'p'), ('b', 'r')]
```

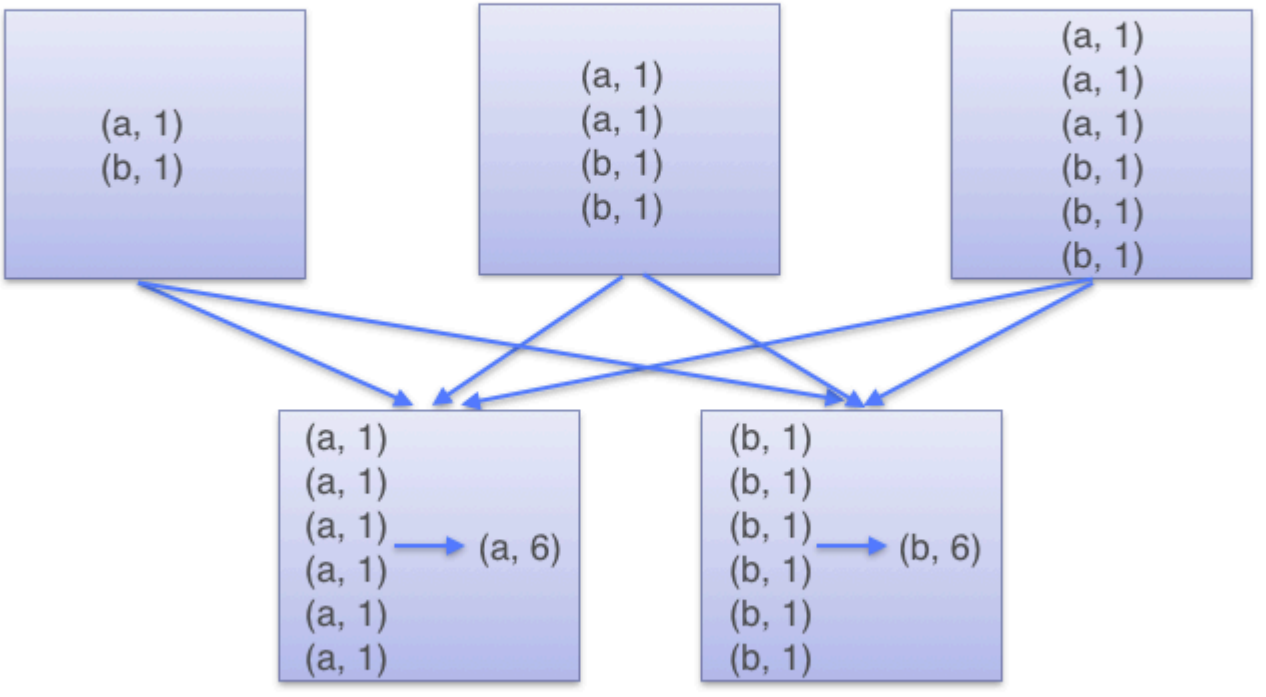
Transformations for a single `PairRDD`

transformation	description
`groupByKey()`	Group values with the same key

Example with `groupByKey`

```
>>> rdd = sc.parallelize([
        ("a", 1), ("b", 1), ("a", 1),
        ("b", 3), ("c", 42)
        ])
>>> rdd.groupByKey().mapValues(list).collect()
[('c', [42]), ('b', [1, 3]), ('a', [1, 1])]
```

GroupByKey


(a, 1)
(b, 1)
(a, 1)
(a, 1)
(b, 1)
(b, 1)
(a, 1)
(a, 1)
(a, 1)
(b, 1)
(b, 1)
(b, 1)
(a, 1)
(a, 1)
(a, 1)
(a, 1)
(a, 1)
(a, 1)
(a, 6)
(b, 1)
(b, 1)
(b, 1)
(b, 1)
(b, 1)
(b, 1)
(b, 6)

Transformations for a single `PairRDD`

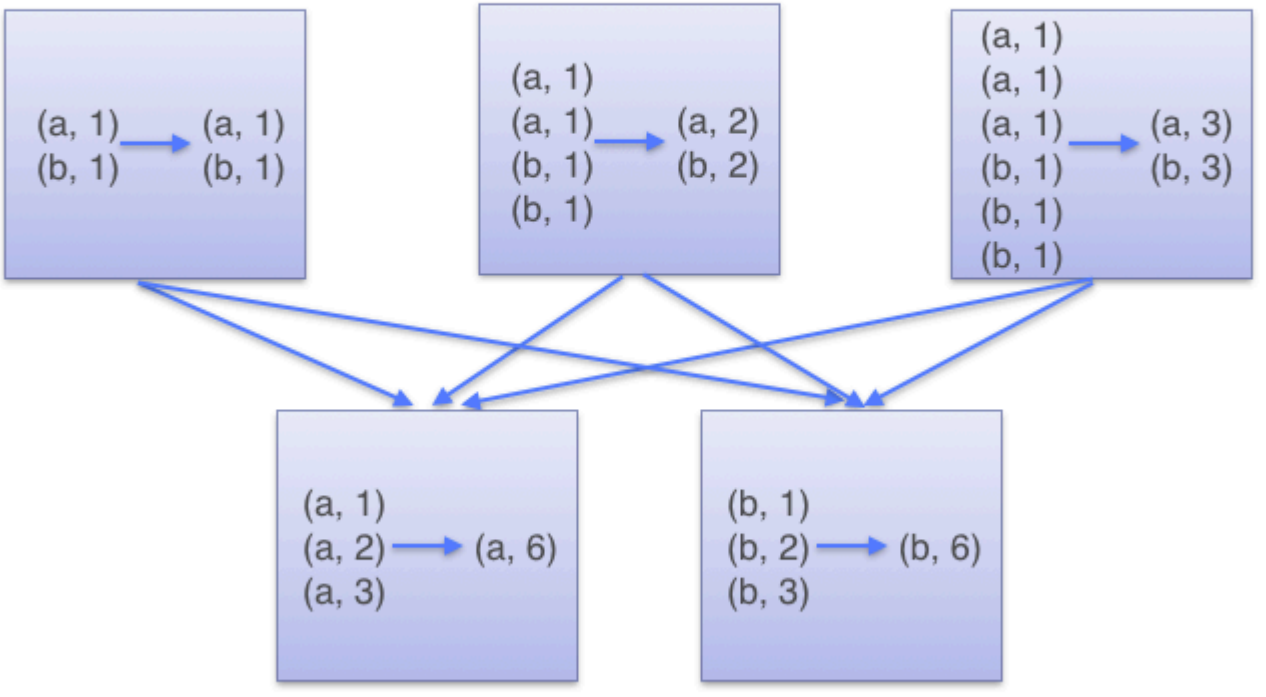
transformation	description
`reduceByKey(f)`	Merge the values for each key using an associative reduce function `f`.
`foldByKey(f)`	Merge the values for each key using an associative reduce function `f`.

Example with `reduceByKey`

```
>>> rdd = sc.parallelize([("a", 1), ("b", 1), ("a", 1)])
>>> rdd.reduceByKey(lambda a, b: a + b).collect()
[('a', 2), ('b', 1)]
```

- The reducing occurs first **locally** (within partitions)
- Then, a shuffle is performed with the local results to reduce globally

ReduceByKey



Transformations for a single `PairRDD`

transformation	description
`combineByKey(createCombiner, mergeValue, mergeCombiners, [partitioner])`	Generic function to combine the elements for each key using a custom set of aggregation functions.

- Transforms an `RDD[(K, V)]` into another RDD of type `RDD[(K, C)]` for a "combined" type `C` that can be different from `V`

The user must define

- `createCombiner` : which turns a `V` into a `C`
- `mergeValue` : to merge a `V` into a `C`
- `mergeCombiners` : to combine two `C`'s into a single one

Transformations for a single `PairRDD`

transformation	description
`combineByKey(createCombiner, mergeValue, mergeCombiners, [partitioner])`	Generic function to combine the elements for each key using a custom set of aggregation functions.

In this example

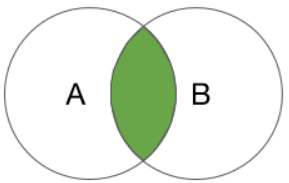
- `createCombiner` : converts the value to `str`
- `mergeValue` : concatenates two `str`
- `mergeCombiners` : concatenates two `str`

```
>>> rdd = sc.parallelize([('a', 1), ('b', 2), ('a', 13)])
>>> def add(a, b):
        return a + str(b)
>>> rdd.combineByKey(str, add, add).collect()
[('a', '113'), ('b', '2')]
```

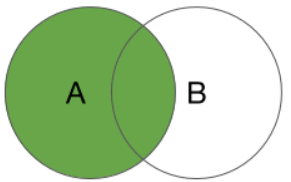
Transformations for two `PairRDD`

transformation	description
`subtractByKey(other)`	Remove elements with a key present in the **`other`** RDD.
`join(other)`	Inner join with **`other`** RDD.
`rightOuterJoin(other)`	Right join with **`other`** RDD.
`leftOuterJoin(other)`	Left join with **`other`** RDD.

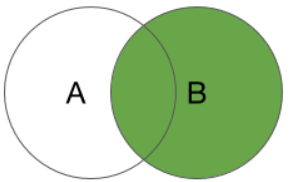
- Right join: the key must be present in the first RDD
- Left join: the key must be present in the **`other`** RDD



INNER JOIN



LEFT OUTER JOIN



RIGHT OUTER JOIN

Transformations for two `PairRDD`

- Join operations are mainly used through the high-level API: `DataFrame` objects and the `spark.sql` API
- We will use them a lot with the high-level API (`DataFrame` from `spark.sql`)

`[Let's go to notebook05_sparkrdd.ipynb]`

Actions for two `PairRDD`

action	description
`countByKey()`	Count the number of elements for each key.
`lookup(key)`	Return all the values associated with the provided `key`.
`collectAsMap()`	Return the key-value pairs in this RDD to the master as a Python dictionary.

`[Let's go to notebook05_sparkrdd.ipynb]`

Data partitionning

- Some operations on `PairRDD`s, such as `join`, require to scan the data **more than once**
- Partitionning the RDDs **in advance** can reduce network communications
- When a key-oriented dataset is reused multiple times, partitionning can lead to performance increase
- In `Spark`: you can **choose which keys will appear on the same node**, but no explicit control of which worker node each key goes to.

Data partitionning

In practice, you can specify the number of partitions with

```
rdd.partitionBy(100)
```

You can also use a custom partition function **hash** such that **hash(key)** returns a hash

```
import urlparse

>>> def hash_domain(url):
        # Returns a hash associated to the domain of a website
        return hash(urlparse.urlparse(url).netloc)

rdd.partitionBy(20, hash_domain) # Create 20 partitions
```

To have finer control on partitionning, you must use the Scala API.